

Tutorial Sheet 5

Synchronization

AUTHOR

Operating Systems Teaching and Research Unit

PUBLISHED

24.06.2024

Task 1: Semaphore

Using semaphore, implement the following synchronization mechanisms.

1. Mutex

Define a structure `struct Mutex` and implement the following functions:

```
1 // initialize your struct Mutex
2 void mutex_init(struct Mutex *mutex);
3
4 void mutex_lock(struct Mutex *mutex);
5
6 void mutex_unlock(struct Mutex *mutex);
```

1. Barrier

Define a structure `struct Barrier` and implement the following functions:

```
1 // initialize your struct Barrier
2 void barrier_init(struct Barrier *barrier, int n_threads);
3
4 // wait until all the threads are waiting, then release the threads
5 void barrier_wait(struct Barrier *barrier);
```

Mutex

```

1  struct Mutex {
2      sem_t mutex;
3  };
4
5  void mutex_init(struct Mutex *mutex) {
6      sem_init(&mutex->mutex, 0, 1); // Initially unlocked
7  }
8
9  void mutex_lock(struct Mutex *mutex) {
10     sem_wait(&mutex->mutex);
11 }
12
13 void mutex_unlock(struct Mutex *mutex) {
14     sem_post(&mutex->mutex);
15 }

```

Barrier

```

1  struct Barrier {
2      int count;
3      int n_threads;
4      sem_t mutex;
5      sem_t barrier;
6  };
7
8  // initialize the barrier for later use
9  void barrier_init(struct Barrier *barrier, int n_threads) {
10     barrier->count = 0;
11     barrier->n_threads = n_threads;
12     sem_init(&barrier->mutex, 0, 1); // Mutex semaphore initially unlocked
13     sem_init(&barrier->barrier, 0, 0); // Initially locked
14 }
15
16 // Barrier to ensure all threads wait here before being released
17 void barrier_wait(struct Barrier *barrier) {
18     sem_wait(&barrier->mutex); // Take the mutex
19     barrier->count++; // Only 1 thread can increase the shared variable at a time
20
21     if (barrier->count == barrier->n_threads) {
22         // The last thread calling "wait" will enter this if-condition, unlock
23         // one another thread, which will in its turn unlock another thread...
24         // until all are unlocked
25         sem_post(&barrier->mutex);
26         sem_post(&barrier->barrier);
27         return;
28     }
29
30     sem_post(&barrier->mutex);
31
32     sem_wait(&barrier->barrier);
33     sem_post(&barrier->barrier);
34 }

```

Task 2: Mensa Ahornstraße

a)

In this task, we simulate the functionality of the Mensa Ahornstraße. We assume there are only two dishes, “Klassiker” and “Vegetarian”. At the counter, both dishes are available. Only one person can be at the counter at a time, stating their food request. If the dish is available, a portion is handed over, the person pays, and then leaves the counter. Otherwise, the person waits at the counter until the dish is available. Whenever the kitchen prepares new portions, they are made available by the cafeteria staff.

Here is the current solution. The `arrivalStudent` function is started by different processes, `fillKlassiker` and `fillVegetarian` are called by only one process.

```
1  #define KLASIKER 0
2  #define VEGETARIAN 1
3
4  int studentsAtCounter = 0;
5  int klassiker = 0;
6  int vegetarian = 0; // shared variables
7
8  void fillKlassiker(int n) {
9      klassiker += n;
10 }
11
12 void fillVegetarian(int n) {
13     vegetarian += n;
14 }
15
16 void arrivalStudent(int wish) {
17     while (studentsAtCounter > 0);
18     studentsAtCounter++;
19     if (wish == KLASIKER) {
20         while (klassiker == 0) {
21             // wait
22         }
23         klassiker--;
24     }
25     else if (wish == VEGETARIAN) {
26         while (vegetarian == 0) {
27             // wait
28         }
29         vegetarian--;
30     }
31     studentsAtCounter--;
32 }
```

Explain the term busy waiting in the context of this task and give two examples from the pseudocode.

b)

Synchronize the solution using mutex. Justify the placement of the critical sections.

c)

Instead of a mutex, synchronize the solution using semaphore. Justify the placement of the critical sections. You may need additional variables for correct synchronization.

Solution

a)

Busy waiting occurs when a process continuously checks a condition without performing productive work. This results in high CPU usage without accomplishing any useful task.

Two examples from the pseudocode:

1. `while (studentsAtCounter > 0);` : The process repeatedly checks the condition until no other student is at the counter.
2. `while (klassiker == 0);` or `while (vegetarian == 0);` : The process repeatedly checks the condition until the desired dish is available.

b)

```
1  #define KLASSIKER 0
2  #define VEGETARIAN 1
3
4  int studentsAtCounter = 0;
5  int klassiker = 0;
6  int vegetarian = 0; // shared variables
7  Mutex counterLock; // initialize mutex
8  Mutex klassikerLock; // initialize mutex
9  Mutex vegetarianLock; // initialize mutex
10
11 void fillKlassiker(int n) {
12     mutex_lock(&klassikerLock);
13     klassiker += n;
14     mutex_unlock(&klassikerLock);
15 }
16
17 void fillVegetarian(int n) {
18     mutex_lock(&vegetarianLock);
19     vegetarian += n;
20     mutex_unlock(&vegetarianLock);
21 }
22
23 void arrivalStudent(int wish) {
24     // ensure only 1 student at a time
25     mutex_lock(&counterLock);
26     studentsAtCounter++;
27     if (wish == KLASSIKER) {
28         while (1) {
29             mutex_lock(&klassikerLock);
30             if (klassiker > 0) {
31                 klassiker--;
32                 mutex_unlock(&klassikerLock);
33                 break;
34             }
35             mutex_unlock(&klassikerLock);
36         }
37     } else if (wish == VEGETARIAN) {
38         while (1) {
39             mutex_lock(&vegetarianLock);
40             if (vegetarian > 0) {
41                 vegetarian--;
42                 mutex_unlock(&vegetarianLock);
43                 break;
44             }
45             mutex_unlock(&vegetarianLock);
46         }
47     }
48     studentsAtCounter--;
49     mutex_unlock(&counterLock);
50 }
```

Justification:

The critical sections are placed around the code that modifies shared variables. This ensures that no two processes can simultaneously modify `studentsAtCounter`, `klassiker`, or `vegetarian`, preventing race conditions.

Notice that this solution still relies on busy waiting, but no more race condition is possible thanks to mutexes.

c)

```
1  #define KLASIKER 0
2  #define VEGETARIAN 1
3
4  Semaphore klassikerSem(0); // initialize semaphore to 0: no klassiker available at start
5  Semaphore vegetarianSem(0); // initialize semaphore to 0: no vegetarian available at start
6  Semaphore counter(1); // initialize semaphore to 1: one place available at the counter
7
8  void fillKlassiker(int n) {
9      for (int i = 0; i < n; i++) {
10         sem_post(&klassikerSem);
11     }
12 }
13
14 void fillVegetarian(int n) {
15     for (int i = 0; i < n; i++) {
16         sem_post(&vegetarianSem);
17     }
18 }
19
20 void arrivalStudent(int wish) {
21     // ensure only 1 student at a time at the counter
22     sem_wait(&counter);
23     if (wish == KLASIKER) {
24         sem_wait(&klassikerSem);
25     } else if (wish == VEGETARIAN) {
26         sem_wait(&vegetarianSem);
27     }
28     sem_post(&counter);
29 }
```

Justification:

Semaphores are used to manage the availability of dishes. `sem_post` is called whenever new portions are added, and `sem_wait` ensures a student waits if the dish is unavailable.

This prevents busy waiting because semaphores rely on signaling other waiting threads upon incrementing the semaphore.

Task 3: Deadlock

Given three resources: hard drive, card reader, and printer. Each process always needs exactly two of these resources exclusively. The following sequences for processes have been defined. All semaphores (hardDrive, cardReader, printer) are mutexes, initialized to 1. Does this solution work? If you find an error or problem, describe how to fix it.

Process A:

```
1 while (condition) {  
2     sem_wait(hardDrive);  
3     sem_wait(cardReader);  
4     useDevices();  
5     sem_post(hardDrive);  
6     sem_post(cardReader);  
7 }
```

Process B:

```
1 while (condition) {  
2     sem_wait(printer);  
3     sem_wait(hardDrive);  
4     useDevices();  
5     sem_post(printer);  
6     sem_post(hardDrive);  
7 }
```

Process C:

```
1 while (condition) {  
2     sem_wait(cardReader);  
3     sem_wait(printer);  
4     useDevices();  
5     sem_post(cardReader);  
6     sem_post(printer);  
7 }
```


Solution

Deadlock can occur if all three processes execute their first wait.

- Process A takes hardDrive
- Process B takes printer
- Process C takes cardReader
- Process A is waiting on cardReader
- Process B is waiting on hardDrive
- Process C is waiting on printer

One solution is to define an order to take the locks. The order should be respected by all processes. For instance, hardDrive -> printer -> cardReader.

Process A:

```
1 while (condition) {
2     sem_wait(hardDrive);
3     sem_wait(cardReader);
4     useDevices();
5     sem_post(hardDrive);
6     sem_post(cardReader);
7 }
```

Process B:

```
1 while (condition) {
2     // switch the locks to respect the order
3     sem_wait(hardDrive);
4     sem_wait(printer);
5     useDevices();
6     sem_post(hardDrive);
7     sem_post(printer);
8 }
```

Process C:

```
1 while (condition) {
2     // switch the locks to respect the order
3     sem_wait(printer);
4     sem_wait(cardReader);
5     useDevices();
6     sem_post(printer);
7     sem_post(cardReader);
8 }
```

No more dead lock.

Task 4: Sleeping Barber

In this problem, we have a single barber working in a barbershop. The barbershop has a waiting room that can hold up to 20 customers. Only one customer at a time can be in the barber's chair for a haircut. If there

are no customers in the shop, the barber, who often gets tired from working too much, will go to sleep. However, if a customer enters the barbershop and finds the barber sleeping, the customer will wake up the barber.

You need to implement the routines for both the barber and the customers, using proper synchronization mechanisms to ensure the requirements are met.

Requirements:

1. The barber should go to sleep if there are no customers waiting.
2. A customer should wake up the barber if they find him sleeping.
3. Only one customer should be in the barber's chair at a time.
4. The waiting room should not hold more than 20 customers at a time.

Questions:

1. Identify the constants that need to be declared.
2. Determine the number of semaphores required and their initial values.
3. Specify the number of mutexes required.

Your implementation should ensure proper synchronization between the barber and the customers, adhering to the constraints mentioned above.

Solution

Constants

```
1 struct barbershop {
2     int chairs = 20;
3     int waiting; // number of people in the waiting room
4     sem_t customers; // initialized to 0
5     sem_t barbers; // initialized to 0
6     sem_t mutex; // initialized to 1
7 }
```

Barber routine

```
1 void barber_routine(struct barbershop *bs) {
2     while (true) {
3         sem_wait(&bs->customers); // if no customer, we sleep and wait
4                                     // if one customer, we take the semaphore
5
6         sem_wait(&bs->mutex);
7         bs->waiting--;           // atomic change to waiting
8         sem_post(&bs->mutex);    // release the mutex
9
10        sem_post(&bs->barbers); // the barber is available for the customer
11        cut_hair(bs);          // we can cut the hair of the customer
12    }
13 }
```

Customer routine

```
1 void customer_routine(struct barbershop *bs) {
2     sem_wait(&bs->mutex);
3     if (bs->waiting < bs->chairs) { // no chair available? leave
4         bs->waiting++;             // one more customer is waiting
5         sem_post(&bs->customers);
6
7         sem_post(&bs->mutex);       // release the mutex
8         sem_wait(&bs->barbers);    // wait until the barber is available
9         get_haircut(bs);          // get the haircut
10    } else {
11        sem_post(&bs->mutex);       // if we're leaving, we release the mutex
12    }
13 }
```